

Projektbeskrivning

Kalender

2024-03-22

Projektmedlemmar:

Mhd. Khaled Helwany <mhdhe919@student.liu.se>

Handledare:

Thea Antonson <thean981@liu.se>

Innehåll

1. Introduktion till projektet.....	2
2. Ytterligare bakgrundsinformation	2
3. Milstolpar.....	2
4. Övriga implementationsförberedelser	4
5. Utveckling och samarbete	4
6. Implementationsbeskrivning	6
6.1. Milstolpar.....	6
6.2. Dokumentation för programstruktur, med UML-diagram.....	6
7. Användarmanual	7

Projektplan

1. Introduktion till projektet

Jag ska utveckla ett kalenderprogram där användaren kan boka in möten och övervaka sitt schema för att hålla sig organiserad och planera i förväg för viktiga händelser. Kalendern ska vara enkel att använda och ge användaren några livskvalitets-features som exempelvis påminnelser, PDF-formatnedladdning och ett fint grafiskt gränssnitt. Detta är självklart preliminär just nu och kan utökas senare under utvecklingen av programmet i mån av tid. Jag har utgått från inspirationsprojektet kalenderprogram och planerar att bygga på förslag från inspirationen.

2. Ytterligare bakgrundsinformation

En kalender är ett mycket relevant verktyg genom mänsklighetens historia som hjälpte oss att hålla reda på dagar, anteckna viktiga datum och sätta en plan för framtiden. Nuförtiden upprätthåller kalendern fortfarande samma funktioner och är en viktig del av samhället. Vare sig det skulle vara att ordna en fest eller att boka ett viktigt arbetsmöte är det alltid två viktiga saker som måste bestämmas först, datum och tid. Så vad är bättre att använda än en kalender, en person som inte planerar för framtiden är med största sannolikhet avsedd att misslyckas. Kalendern har haft en lång och fruktbar historia som lede till många olika typer av kalendrar, alla med olika egenskaper. För de som är intresserad av att veta mer kan ni klicka på följande länk! : <https://en.wikipedia.org/wiki/Calendar>

3. Milstolpar

#	Beskrivning
1	Skapa en grundläggande struktur för tidsperioder med hjälp av timmar och minuter. En egen klass för minuter, timmar, tidsrymd som kombinerar minuter och timmar och sist en klass för tidsperioder som kombinerar 2 tidsrymder för att skapa grunden för bokning av tid som senare ska implementeras
2	Klasser för dagar, veckor, månader och år är färdiga. Designa dessa klasser för att interagera med varandra för att bilda kalenderstrukturen och datumklassen. Kombinera dessa klasser tillsammans med tidsperiodsklassen, en titel och ett datum för att skapa en Appointment klass alltså möte som man kan boka in i kalendern.
3	Kalenderklassen är klar med bokningsfunktionaliteten. Denna klass ska använda sig av Appointment klassen mest. Användaren ska också kunna se Kalendern ska också kunna visas och användaren ska ha möjligheten att välja en specifik månad för att kolla på.

- 4 Gränssnittet ska vara nästan helt färdigt åtminstone för ett grundläggande kalenderprogram där användaren kan klicka på knappar i stället att anropa funktioner själv. Funktionalitet för att kunna se möten för ett visst datum ska också vara klar och implementerat i gränssnittet.
- 5 Funktionalitet för att spara kalendern som en PDF-fil och ladda en sparad kalender. Bygga på val av datum funktionaliteten så att användaren kan då i stället välja att ta upp alla möten för en viss tidsperiod.
- 6 Lägg till en funktion för att ta bort möten. de borttagna mötena kommer att läggas till sparade av programmet vilket gör det möjligt för användaren att ångra borttagningen i fall av en oavsiktlig radering eller omställning
- 7 Ha koll vid vecka 16 att koden är körbar och skapar en fungerande kalender med några funktionaliteter.
- 8 Felhantering och inkapsling av koden hittills. Granska alla fält och metoder och avgör om det är bäst att ha dem public, private, static o.s.v.
- 9 Notiser när det är en timme kvar till början av ett möte. Användaren ska kunna välja om hen vill ha notiser eller inte.
- 10 Visa flera kalendrar: Många har flera olika kalendrar som de vill visa samtidigt. Det kan till exempel vara en privat och en för jobbet. För att det ska gå att få en bra översikt krävs det dock att man kan visa alla kalendrar samtidigt. Det syftar den här milstolpen till att göra möjligt.
- 11 Exportera till iCalendar: Om användaren vill att andra ska kunna ta del av ens kalender är det bra att kunna exportera till iCalendar format eftersom de flesta kalenderprogram har stöd för det formatet. OBS! att manuellt generera en iCalendar fil kan vara onödigt arbete, sök i stället efter existerande bibliotek
- 12 Låt andra följa kalendern: Om en annan användare importerar en iCalendar fil kan de importera din kalender men hen kommer inte få en uppdaterad kalender om man ändra något. Såvida man inte exporterar en kalender en gång till och skickar den på nytt. För att den andra användaren ska få en uppdaterad version av kalendern kan man låta dem följa sin kalender. Det innebär att ert program måste lyssna på nätverket och behandla inkommande förfrågningar efter kalendrar. För att implementera det här behöver man läsa på om socketprogrammering i Java
- 13 Extra funktionaliteter som man kan komma på kanske senare om det finns tid kvar och polering av koden

4. Övriga implementationsförberedelser

Varje klass ska ha getters och en konstruktor. En klass för varje tidsenhet och vissa klasser ska kombinera 2 eller flera tidsenheter exempelvis datum klassen tar in en dag, månad och år. Själva kalender klassen kan vara helt enkelt en samling av appointments i en lista kanske, ska undersökas vidare vilket format är bäst under utvecklingen.

Biblioteket java. Time kommer att användas för att skapa en mer effektiv struktur. När grundläggande strukturen för kalendern är färdig kan andra funktionaliteter implementeras. Book ska addera en appointment till kalendern, remove tar bort en appointment och sparar den i en lista av raderade appointments för senare implementation av en undo troligtvis. Show metoden kommer att förändras ganska mycket under utvecklingen, först ska den visa en viss vecka, sedan ett datum och sist en tidsperiod. Det är inte en dålig idé att kunna ha alla dessa alternativ tillsammans i programmet, kanske en show metod för varje alternativ? (showWeek(), showDate() och showPeriod()). Att spara, nedladda och visa upp en sparad kalender tänker jag att utföra i PDF formatet och för det ska jag använda ett bibliotek som hanterar just PDF filer, Apache PDFBox är ett förslag för det. Notiser tänkte jag att använda javas Timer och TimerTask klasser för att implementera det.

5. Utveckling och samarbete

- Vilka tider tänker ni jobba? Går ni till alla resurstillfällen eller bara vissa, och hur kommer ni i så fall överens om vilka? Vid vilka tider arbetar ni tillsammans med projektet utöver resurstillfällena? Jobbar ni ibland (tillsammans) på kvällar eller helger? Är någon av er ofrånkomliga upptagen vid vissa tider och hur löser ni detta? :Jag kommer att arbeta under laborations tillfällen, i själva labbsalarna eller på valfri plats. Jag kommer också att jobba med projektet under egen tid också, kanske 2 eller 3 timmar per vecka från egen tid.
- När ska ni ha kommit till en viss punkt i projektet? Vill ni sätta upp en egen (enkel) tidslinje för vad ni vill bli klara med och när? :Jag ska kolla på projektets progression vid halvvägs punkten av tiden vi får för utvecklingen alltså vecka 16.
- Vad är ett godtagbart skäl för att inte kunna komma till ett "arbetsmöte" som redan har bokats? : Jag tycker att känsliga ärenden eller plötsliga hinder är skäl nog för att missa ett viktigt möte. Exempelvis en trafikolycka eller sjukhusvistelse av en familjemedlem

Projektrapport

6. Implementationsbeskrivning

I det följande avsnittet beskrivs implementeringen av koden och dess roll i att skapa ett välskrivet kalenderprogram som är intuitivt för användaren. Med stöd av UML-diagram och tydliggörande av specifika komponenter, blir detta projekt lättanvänt och anpassningsbart för alla som väljer att dra nytta av det.

6.1. Milstolpar

Ms 1: genomfört helt

Ms 2: genomfört helt

Ms 3: genomfört helt

Ms 4: genomfört helt

Ms 5: genomfört delvis (json fil i stället för PDF och enskilda dagar kan visas)

Ms 6: genomfört helt

Ms 7: genomfört helt

Ms 8: genomfört helt

Ms 9: genomfört helt

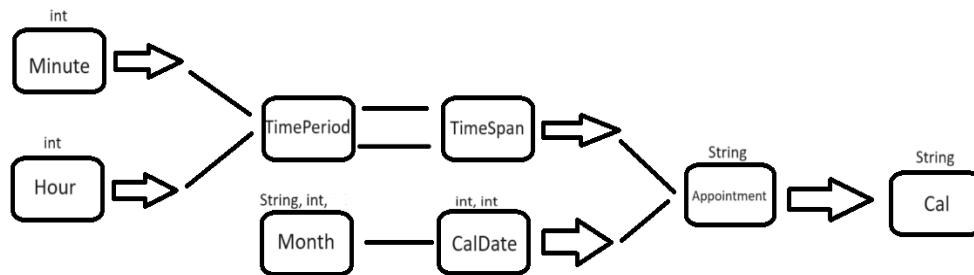
Ms 10: genomfört ej

Ms 11: genomfört ej

Ms 12: genomfört ej

Ms 13: genomfört delvis (interaktivt rutnät)

6.2. Dokumentation för programstruktur, med UML-diagram



UML-Diagram 1

Ett grundläggande UML-diagram för att illustrera den övergripande strukturen av källkoden. Varje klass representeras av en rektangel, med program flödet visualiserat genom linjer och pilar. Standardprogrammeringstyper såsom int och String används inom vissa klasser och anges tydligt ovanför klassens rektangel.

UML-diagrammet ovan ger en grundläggande översikt av projektet. Klasserna Minute och Hour representerar minuter och timmar inom programmet. En int-datatype används för att definiera dem, och båda klasserna har en konstruktor- och getter-metod. Därefter används Minute- och Hour-datatypeerna för att skapa tidsenheter i kalendern, som sedan tillsammans med en annan TimePeriod bildar tidsintervallet TimeSpan. Både TimePeriod och TimeSpan har standardgetters och en konstruktor. Särskilda metoder implementeras i TimePeriod och TimeSpan för att bistå programmet med att upptäcka överlappande möten. Metoden periodPrecedes jämför två tider för att avgöra om den första tiden kommer före eller efter den andra. Denna metod används sedan i overlap-metoden för TimeSpan, som varnar för överlappande tidsintervall. Den kontrollerar om början (start) av det första tidsintervallet förekommer slutet av det andra tidsintervallet (end) och vice versa. Om dessa krav är uppfylla indikerar det en överlapp, och programmet tar sedan lämpliga åtgärder i senare klasser.

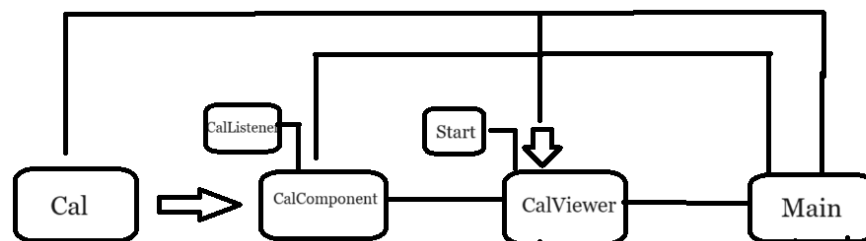
Nästa datatype är Month, som är av grundläggande betydelse. Month innehåller tre Java-mappar som innehåller viktig information om årets månader, såsom namnet, antalet dagar i månaden och månadens nummer i kalendern. Dessa mappar nås genom tre motsvarande metoder som har en standardutgång (default) om det inmatade värdet inte hittas, vilket undviker krascher. En intressant detalj är att dessa metoder och mappar använder datatypen Integer i stället för standard int för att dra nytta av dess flexibilitet som lagrat värde. Denna information kommer att vara mycket användbar i senare delar av projektet för att strukturera användargränssnittet (GUI) och övervaka programmets framsteg. Datatypen Month används även för att skapa datumet CalDate. Dagen och året representeras som vanliga int eftersom det ansågs överflödigt med en egen klass för dem. Eftersom beslutet togs att kalendern ska byggas med ett månatligt perspektiv, kan dagen och året i CalDate representeras perfekt med enbart int-datatyper. Slutligen kommer Appointment-datatypen, som skapas av en CalDate, en TimeSpan och en String för att representera ämnet. De fyra klasserna TimePeriod, TimeSpan, CalDate och Appointment har alla någon form av åsidosatta (override) metoder för equals och toString för att implementera unika jämförelser och sätt att skriva ut dessa datatyper, vilket kommer att vara mycket användbart för GUI-delen av kalendern.

I slutet av UML-diagram 1 återfinns huvudklassen i detta projekt, som representerar en kalender. Den heter bekvämt Cal för att undvika förväxlingar med populära Java-bibliotek. Cal-klassen är i huvudsak en namngiven lista av Appointment-klassen, vilket i grunden är vad en riktig kalender är, en samling noterbara händelser. Många viktiga och komplexa metoder har lagts till denna klass för att uppnå önskad funktionalitet.

Först finns showDay-metoden, som tar tre int-inmatningar - en dag, ett månadnummer och ett år - och sedan itererar genom alla Appointment-objekt i listan 'appointments', och undersöker om de har matchande datum med de inmatade datumen. Resultatet skickas sedan till GUI-delen. Metoden använder sig av en hjälpmetod, dayAppointments, för att utföra det tunga arbetet med att jämföra och samla möten för en viss dag i en lista.

Därefter kommer metoderna book och remove, som är ganska lika men utför helt motsatta uppgifter. Båda metoderna tar emot en datum-inmatning och tidsintervaller som int-datatype. Skillnaden är att book behöver två tidsintervaller för att skapa en tidsrymd för bokningen, medan remove endast behöver ett intervall för att identifiera möten att ta bort. Metoderna använder sig av en hjälpmetod, requirements, för att identifiera felaktiga inmatningar, till exempel år = 6. Eftersom remove tar emot färre argument har en konstant, IGNORE, skapats i Cal-klassen för att ignorera endHour och endMinute-argumenten. Denna konstant kan användas i framtida metoder för att hoppa över irrelevanta krav. När inmatningarna har passerat testerna kan metoderna utföra åtgärder med dem; annars hanterar programmet felet med ett felmeddelande till användaren.

Slutligen kallar metoderna sort, som sorterar möteslistan 'appointments' med hjälp av en jämförelseobjekt från Comparator-biblioteket. Den sorterar listan från tidigast till senast genom att jämföra tidsenheter från störst (år) till minst (minut). Efter att sorteringen är klar, anropas metoden notifyListeners för att meddela GUI-komponenten att en förändring har skett i kalendern och att det är dags att uppdatera. Genom denna struktur skapas en enkel och effektiv hierarki som kommer att utgöra den fasta grunden för detta projekt.



UML-diagram 2

Diagrammet ovan visar hur GUI-delen av projektet är djupt sammanflätad. En pil indikerar en konstruktör instans och en linje betyder att vi bara använder klassen som linjen kom från, till exempel att implementera den som ett gränssnitt (CalListener) eller helt enkelt skapa en lokal instans.

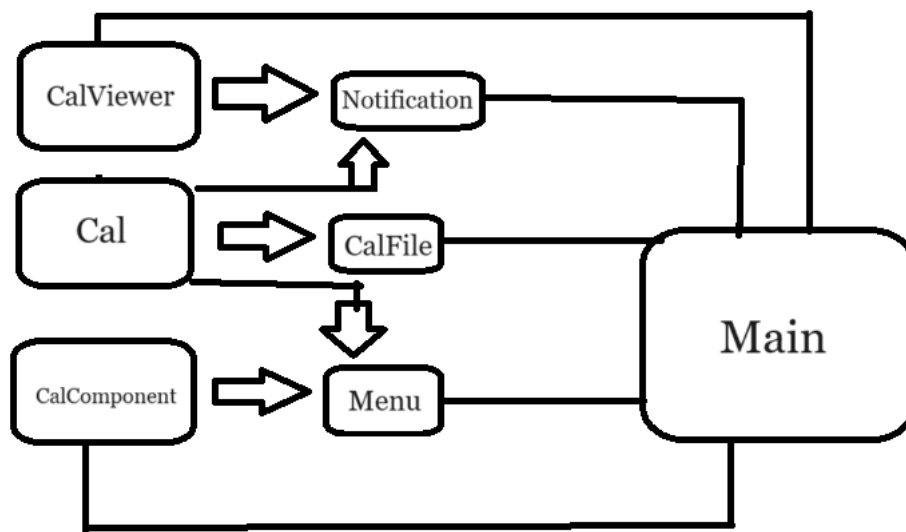
Den grafiska sidan av kalendern är betydligt mer intrikat, vilket visas i UML-diagram 2. I basen av den finns kalenderklassen Cal, som lagrar all viktiga data och vidarebefordrar den till andra delar av programmet när det behövs. Det finns en linjär huvudgren som syns i diagrammet med flera mindre delar, som alla slutligen konvergerar vid Main, klassen som kör kalendern. Huvudgrenen består av Cal, CalComponent, CalViewer och Main.

CalComponent är ansvarig för att rita och uppdatera kalendern för användaren vid start av programmet och vid ändringar i kalendern. Den implementerar gränssnittet CalListener som nämndes i slutet av föregående avsnitt. Denna implementering hjälper programmet att kommunicera en förändring i objektet Cal till klassen CalComponent. Dess konstruktör tar in en Cal-instans och formar unikt hur användarens kalender kommer att se ut baserat på informationen i den instansen. Konstruktorn ställer in en standardlayout och anropar metoder som låter användaren interagera med applikationen, registerButtons och registerMouse. registerButtons använder JButton-datatype och actionlyssnare (ActionListeners) för att skapa knappar i programmet. Knapparnas design, storlek och placering dokumenteras utförligt med

kommentarer i koden. När knapparna "Show", "Remove" och "Book" klickas skapas ett användarinmatningsfönster, vilket också förklaras noggrant i koden för respektive knapp.

Därefter kommer registerMouse-metoden, som gör en viss del av kalenderfönstret klickbar för användaren. Det enklaste sättet att navigera i kalendern är genom ett månadsbaserat gränssnitt där en månad i taget visas med enkel cykling genom pilknapparna. Ett rektangulärt rutnät används för att visa de enskilda dagarna i månaden och eventuella möten som passar i varje rektangel. Metoden registerMouse gör detta rutnät interagerbart för användaren genom att låta dem se alla möten för en specifik dag genom att klicka på motsvarande rektangel. Beräkningarna för det klickbara området för varje rektangel görs baserat på den aktuella visade månaden, som är snyggt lagrad som en privat sträng i klassen. Bredden och höjden på varje komponent i klassen använder konstanterna TILE_WIDTH_FACTOR och TILE_HEIGHT_FACTOR för att göra det enkelt att ändra designen under utveckling.

CalComponent-klassen avslutas med målningsmetoden paintComponent och dess repaint-anropsmetod calChanged, som kallas varje gång CalListener aviseras i Cal-klassen. Rutnätet och all relevant kalenderinformation (kalendernamn, visad månad och år, samt avisering inställning) har alla inställda positioner och storlekar som också använder de tidigare nämnda konstanterna. På grund av storleksbegränsningar i rutnätets rektanglar visas endast de två tidigaste mötena i det begränsade utrymmet. Där kommer det klickbara rutnätet till sin rätt, och användaren får en enkel påminnelse om den funktionen när rektanglarna fylls upp. Bokningsnamnet är också begränsat till 8 tecken innan det ersätts med tre punkter.



UML-diagram 3

Ett diagram som visar hur större klasserna Cal, CalViewer och CalComponent används för att skapa mindre klasser som tillhandahåller stödjande funktionalitet i Main. Pillar indikerar en konstruktor-instans och linjer är för lokala instanser

Innan nästa del av GUI huvudgrenen, CalViewer, kan diskuteras måste några mindre stödjande klasser som tillhandahåller specifik funktionalitet förstås grundligt. Från och med CalFile sparar och läser den här klassen lagrade kalenderfiler med en Cal-instans. Projektet använder filformatet JSON för att lagra kalenderns namn och möten, vilket gör att användaren inte behöver börja om från början varje gång de startar programmet. Med hjälp av Gson-biblioteket skapar och läser metoderna saveCal och

readCal JSON-filer. En berömd spar teknik används i saveCal för att förhindra eventuell dataförlust genom att först spara innehållet på en temporär fil och sedan göra den temporära filen till den nya huvudfilen. På så sätt förblir den ursprungliga huvudfilen orörd tills den inte längre anses vara fördelaktig. Läsning av den lagrade filen görs med readCal, och den använder helt enkelt ett FileReader-objekt för att extrahera kalendernamnet och listan över möten och ställa in den till den Cal-instans som CalFile-klassen fick. Det är värt att notera att båda dessa metoder kastar undantag som hanteras senare i det grafiska användargränssnittet när det är lämpligt.

Nästa diverse klass heter helt enkelt Start, och dess huvuduppgift är att visa en välkomstbild när den anropas. Koden här är relativt enkel och självförklarande, eftersom Start utökar den mycket effektiva JComponent-klassen. Konstruktören tar in en Dimension datatyp, som kommer att användas i paintComponent-metoden för att ange storleken på bilden som laddas från mappen "images" i resurskatalogen.

En annan mindre men viktig klass som detta projekt innehåller är klassen Menu. När den används skapas en menyrad med hjälp av en JFrame. Konstruktorn tar också ett Cal- och CalComponent-objekt för alternativen som gör ändringar i dessa objekt. De nuvarande tillgängliga valen klassificeras i två kategorier, Fil och Redigera. Fil-menyn har menyalternativen att avsluta och aktivera eller inaktivera aviseringar, medan Redigera bara har ett ångra-objekt som anropar ångrametoden i Cal-instansen. CalComponent-objektet uppdateras när en ändring av utseendet på kalendern görs.

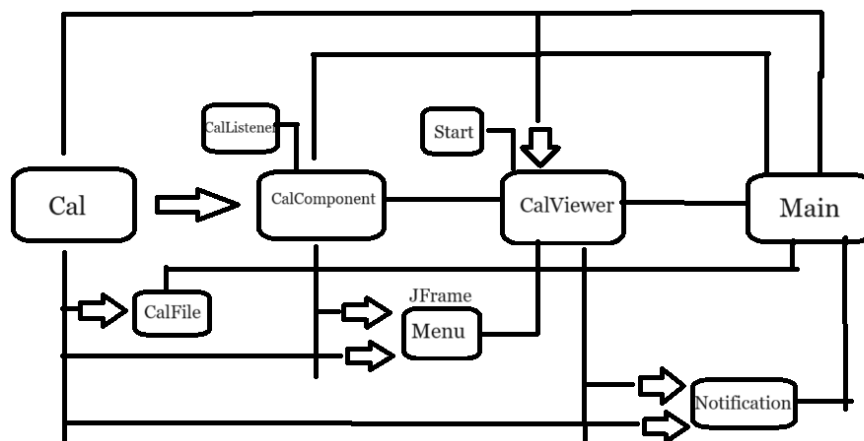
Uppmärksamheten kan nu flyttas mot klassen som kopplar användaren till programmet genom att visa fönster, CalViewer. Ett Cal-objekt krävs för konstruktorn för att visa den korrekta kalendern med alla efterföljande nödvändiga popup-fönster. Huvudmetoden för CalViewer-klassen är show, som kombinerar en JFrame, en CalComponent gjord av Cal-instansen i konstruktorn och de tidigare nämnda stödklasserna Start och Menu för att visa användaren sin egen kalender med alla funktioner för detta program i ett kompakt, rent fönster. En timer används i show-metoden för att visa välkomstskärmen i tre sekunder innan den faktiska kalendern visas. Resten av metoderna i CalViewer visar alla typer av mindre popup-fönster för att komplettera huvudfönstret och undvika röran. Alla är ganska självförklarande förutom den sista metoden displayFileError.

När en FileNotFoundException fångas (vanligtvis när programmet används för första gången) presenteras användaren med två alternativ angående sin kalenderfil, antingen att skapa en ny eller låta programmet försöka hitta en JSON-kalenderfil att läsa. På så sätt kan användaren välja att ladda den sparade kalendern som de har, om de till exempel använder applikationen på en ny enhet. För att uppnå denna funktionalitet för livskvalitet tar metoden displayFileError in en CalFile, ett felmeddelande och en feltyp för att hantera det som gick fel. Feltypen FileNotFoundException låter användaren antingen skapa en ny genom att anropa saveCal eller ladda en giltig fil och försöka igen. Ett godtyckligt antal försök är möjligt med hjälp av en While-loop i Main så att användaren kan prova om och om igen tills en fil hittas och läses framgångsrikt. Det är även i denna metod som IOExceptionen som readCal och saveCal kastar hanteras från Main. Fel typen IO tillhandahålls för metoden, och den varnar användaren om det på lämpligt sätt genom felmeddelanden så att en lösning kan försökas. Det sista värt att nämna är att varje catch-sats följs av en Logger inte bara i CalViewer utan i hela projektet för att hantera fel mer effektivt.

Den sista klassen som behöver förklaras innan GUI-delen kan snyggt knytas ihop är Notification, som ansvarar för hanteringen av notifieringar. En CalViewer-instans kombineras här med en Cal-instans så att programmet meddelar användaren korrekt och endast om användaren vill. Som tidigare nämnts har Menu-klassen ett menyalternativ som låter användaren växla aviseringsinställningen genom att ändra en boolesk variabel (enabled) i Cal-instansen till motsatsen av vad den var. Denna ändring uppmanar sedan till en ommålning i CalComponent så att meddelandeetiketten

uppdateras. En timer i Notification-klassen kontrollerar varje minut om ett meddelande behövs genom att först anropa isEnabled i Cal och sedan fortsätta med resten av kontrollen. Vid en första anblick kan det tyckas att denna timerinstans kan konverteras till en lokal variabel (de flesta utvecklingsmiljöer kommer att påpeka detta). Timerinstansen kommer dock i så fall att gå utanför räckvidden under programkörning vilket kommer att resultera i att den skrotas automatiskt och förhindrar att den fungerar som förväntat. Användaren ska endast meddelas om ett möte är en timme bort från lokal tid. LocalDate-biblioteket används därför här för att göra resten av kontrollen mer effektiv genom att få den aktuella dagen och kontrollera mötena för den dagen enbart i stället för varje enskilt möte i kalendern.

Slutligen, när programmet slutför kontrollen och anser att användaren ska meddelas, anrops displayNotification från CalViewer-instansen och en ljudfil spelas upp med hjälp av JavaZoom-biblioteket. Eventuella fel som ljudsignalen kan orsaka hanteras också med CalViewer och använder Loggers.



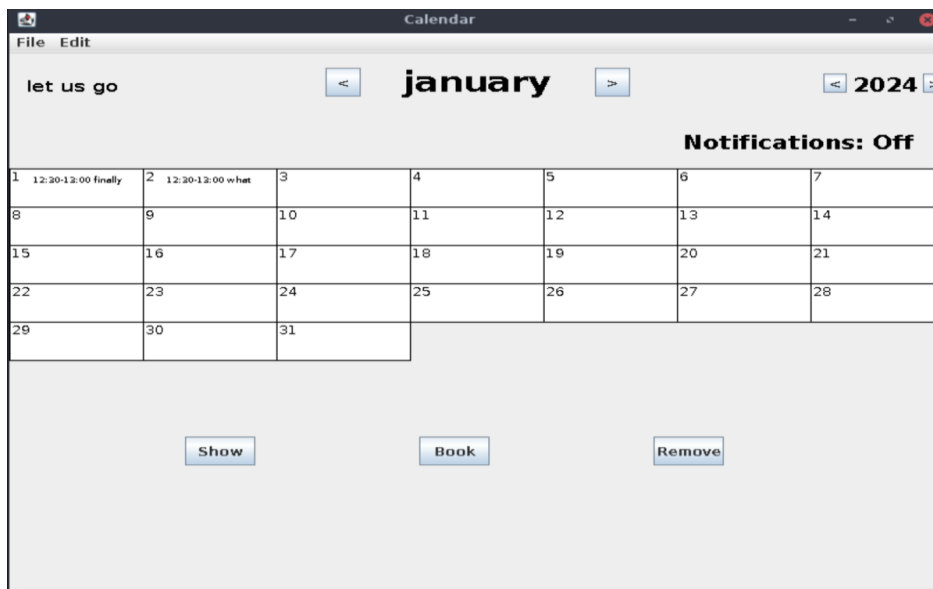
UML-diagram 4

En fullständig översikt över det grafiska användargränssnittet för detta projekt i UML-diagrammet ovan. Pillar indikerar en konstruktor-instans och linjer är för lokala instanser

Förklaringen av dokumentationen kan avslutas nu med kärnan i projektet, Main. Som framgår av UML-diagram 4, konvergerar varje del av det grafiska användargränssnittet, stor som liten, så småningom i hjärtat av programmet, Main. Klassen själv initierar instanser av varje klass vi hittills pratat om och kombinerar dem för att få slutresultatet av ett fungerande kalenderprogram. Som vanligt fångas alla möjliga undantag upp men inga Loggers används här eftersom det redan utförs i själva CalViewer.

7. Användarmanual

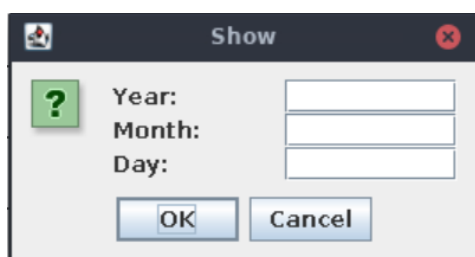
Välkommen till denna korta användarmanual för att komma i gång med kalenderapplikationen. Innan du börjar, se till att du har den senaste versionen av Java installerad för att kunna köra programmet optimalt. När detta är klart, navigera till GitLab-förvaret för att hämta de filer som behövs för att starta din kalender. När filerna är nedladdade kan du fortsätta och starta applikationen. Först och främst, låt oss ta en närmare titt på huvudfönstret som du behöver bekanta dig med.



Figur 1. Kalendern

En enkel menyrad täcker övre delen av fönstret. Den aktuella versionen har alternativ relaterade till filen och redigering av kalendern. Fil-sektionen (File) innehåller alternativ för avstängning och avisering. Det senare alternativet låter dig inaktivera eller aktivera aviseringar (standardinställningen är avstängd, och en indikator visar det aktuella läget). Redigeringsdelen (Edit) innehåller endast ett ångra-alternativ än så länge som låter dig boka om borttagna möten i fall av oavsiktlig borttagning eller omställning. Uppe till vänster på skärmen hittar du namnet på din kalender. I mitten visas den aktuella månaden, med året placerat i det övre högra hörnet av skärmen. Som bilden visar, finns det navigationspilar till vänster och höger om månads- och årsvisningen. Dessa knappar gör det enkelt att navigera i kalendern genom att klicka på vänsterpilen för att gå ett steg bakåt och högerpilen för att gå ett steg framåt. Gränserna för kalendern sträcker sig från det nuvarande året och tio år framåt (mellan 2024 och 2034 när denna manual skrivs).

Under navigeringsdelen av fönstret finns ett rektangulärt rutnät som representerar dagarna i den aktuella månaden, med siffror i varje rektangels hörn för att indikera dagens datum. Slutligen, längst ner i fönstret, finns knapparna som du kommer att använda för att dra nytta av din personliga kalender. Den aktuella versionen av denna kalender har tre knappar: en för att visa möten för ett specifikt datum, en för att ta bort ett möte och en för att boka möten. Låt oss ta en närmare titt på de fönster som visas när dessa knappar klickas.



Figur 2. Show inmatningsfält

När du klickar på "Show"-knappen kommer du att presenteras fönster som visas i figur 2. Därefter måste du skriva in datumet för den önskade dagen med endast siffror. Processen att behöva klicka på en knapp och sedan skriva in datum för att kontrollera varje enskild dag kan vara tröttsam, men det är inte ett problem i detta kalenderprogram. Det rektangulära rutnätet, som diskuterades i det tidigare avsnittet, kan klickas på för att visa alla möten för den dag du valde!

The 'Book' dialog box is a standard Windows-style window with a title bar that says 'Book'. On the left side, there is a green square icon with a white question mark. The main area contains two columns of input fields. The left column has labels 'Year:', 'Month:', 'Day:', and 'Subject:' followed by empty text boxes. The right column has labels 'Start Hour:', 'Start Minute:', 'End Hour:', and 'End Minute:' followed by empty text boxes. At the bottom center, there are two buttons: 'OK' and 'Cancel'.

Figure 3. Book inmatningsfält

Om du klickar på "Book"-knappen kommer fönstret i Figur 3 att visas. Du kommer att bli ombedd att ange all viktig information om ditt möte, inklusive datum, starttid, sluttid och ämne. Observera att alla inmatningar utom ämnet endast måste vara i siffror. När du har bokat mötet kommer det att visas i rätt rektangel beroende på det inmatade datumet tillsammans med mötets tidsperiod och ämne. Det är dock viktigt att nämna att endast de två tidigaste mötena kommer att visas i rektangeln. Om dagen har fler än två möten på grund av storleksbegränsningar, kommer en "click to show more"-dialog att visas.

The 'Remove' dialog box is a standard Windows-style window with a title bar that says 'Remove'. On the left side, there is a green square icon with a white question mark. The main area contains a single column of input fields with labels 'Year:', 'Month:', 'Day:', 'Start Hour:', and 'Start Minute:' followed by empty text boxes. At the bottom center, there are two buttons: 'OK' and 'Cancel'.

Figur 4. Remove inmatningsfält

Sist har vi knappen "Remove", och det resulterande fönstret motsvarar detta val. Du kommer att ombudjas att ange all nödvändig information för att identifiera det specifika mötet du vill ta bort. Liksom tidigare måste dessa inmatningar vara giltiga nummer. Om inga möten hittas kommer programmet att meddela dig.

Manualen ska avslutas med instruktioner om viktiga filplaceringar. Eftersom det här kalenderprogrammet stöder att spara din kalender som en fil, se till att din kalenderfil är av typen json och har namnet "calendar.json". Denna fil bör placeras i katalogen "tdde30-projekt-2024-d1-g21-01". I samma katalog bör mappen "resources" ligga som är avgörande för många implementeringar och därför inte bör flyttas eller modifieras. Om aviseringsljudet inte fungerar korrekt, se till att kontrollera ljudfilen i resurskatalogen så att den placeras i ljudmappen med namnet notification.mp3. Slutligen finns en PDF-fil som innehåller projektets plan och förklaring av koden i katalogen "mhdhe919" för de som är intresserad! Med allt detta sagt, tack för att du tog dig tid att prova den här applikationen och jag hoppas att du hittar riklig användning i den.